Sprint 4 – Technical

Team 8's Submission for Sprint #4

Team Name: Wolf of The 7th Street

Pipeline Contract and Stage Map

Pipeline Contract

1. Project question addressed by this pipeline

Our final project question is whether historical stock-market information can be used to identify patterns and predict whether a stock price will increase on the next trading day.

The Sprint 4 PySpark pipeline prepares minute-level stock data into daily stock data. It also reproduces the monthly stock-price increase analysis from Sprint 3, performs a second monthly analysis using average daily percentage change, and prepares the data for a machine-learning model in Sprint 5.

2. How the result contributes to the final project

The results contribute to the final project in several ways:

- Minute-level observations are converted into daily stock records.
- Daily stock features are created for machine-learning use.
- The PA3 monthly increase-ratio analysis is reproduced and verified.
- A second analysis investigates whether stock behavior changes by calendar month.
- Training and testing datasets are prepared for the Sprint 5 prediction model.
- PySpark and DuckDB implementations are compared for correctness and speed.

3. Raw input file, delimiter, header status, and fields used

Raw input: APOLLOHOSP minute-level stock data from the PA3 sample.

Delimiter: Tab-separated values.

Input size: 1,033,587 rows.

Fields used:

- name

- date
- open
- high
- low
- close
- volume
- price_change
- volume_change

The sample data was used to prepare and test the PySpark pipeline before applying the same process to larger data.

4. Expected schema of the daily analysis-ready table

One row represents one stock on one trading day.

The daily table uses (name, date) as its natural key.

Column	Data Type	Description
name	String	Stock ticker symbol
date	Date	Trading date
open	Double	Daily opening price
high	Double	Highest price during the day
low	Double	Lowest price during the day
close	Double	Daily closing price
price_change	String	Whether the stock increased compared with the previous day
change_pc	Double	Daily percentage range calculated as (high - low) / open

5. Derived variables

The pipeline creates the following derived variables:

change_pc:

```
None
(high - low) / open
```

price_change:

The current day is compared with the previous day to determine whether the price increased or decreased.

The first day has no previous trading day, so its price-change value is recorded as 0.

6. Rules

Filtering:
For Analysis 1, only rows with increased or decreased price labels are included. Rows with price_change = None are excluded from the comparable-row calculation.

Missing values:
Rows with missing price_change values are retained during preparation but excluded from the increase-ratio analysis.

Grouping:
Daily stock records are grouped by calendar month for the two analytical questions.

Ordering:
Analysis results are ordered by month number from 1 through 12.

Join:
Analysis 2 uses a month lookup table to add the month name and quarter. The lookup keys are unique, no daily rows were unmatched, and the row count remained 2,769 before and after the join.

Window:
A previous-day comparison is used to create price_change. The window is ordered by trading date. The first row has no previous day and is assigned the initial value of 0.

Stage Map

```
None
Minute-level input data
        ↓
Read and inspect schema
        ↓
Cast fields and validate values
        ↓
Aggregate minute records into daily records
        ↓
Create change_pc and price_change
        ↓
Save prepared data as Parquet
        ↓
Read Parquet back and verify schema/count
        ↓
Analysis 1: Monthly increase ratio
        ↓
Analysis 2: Monthly average change_pc
        ↓
Create training, testing, and development datasets
        ↓
Compare PySpark and DuckDB benchmark results
        ↓
Sprint 5 machine-learning handoff
```

Data Preparation and Quality Checks

The PySpark notebook read the minute-level data, inspected its schema, selected the required columns, converted fields to appropriate types, and aggregated the records by trading date.

The final daily table contained 2,769 rows.

The following quality checks were completed:

- Minute rows read: 1,033,587
- Minute rows rejected: 0
- Valid minute rows: 1,033,587
- Rows with null model values: 0
- Invalid negative values: 0

- Daily rows before domain filtering: 2,769
- Daily rows after domain filtering: 2,769
- Date key uniqueness: True
- (name, date) uniqueness: True
- Duplicate month lookup keys: 0
- Unmatched daily rows after the month join: 0
- Parquet read-back row count: 2,769
- Parquet schema verification: True
- low <= high: True
- low <= open <= high: True
- low <= close <= high: True

The PA3 validation rules rejected zero minute rows. There were 73,697 valid rows with price_change = None, which were excluded from Analysis 1. There were 959,890 comparable rows containing increased or decreased labels.

The issue counts were recorded separately because some data-quality conditions could overlap. Therefore, the counts were not added together as if they represented completely separate rows.

Parquet Creation Process

The prepared daily dataset was saved as Parquet under:

```
None
pa4/data/prepared/daily_full.parquet
```

The training dataset was saved under:

```
None
pa4/data/prepared/train.parquet
```

The testing dataset was saved under:

```
None
pa4/data/prepared/test.parquet
```

The development sample was saved under:

```
None
pa4/data/prepared/dev_sample_train.parquet
```

The Parquet files were read back into PySpark. The row counts and schemas were verified after reading.

The following read-back checks passed:

- Daily Parquet row count: 2,769
- Training Parquet row count: 2,129
- Testing Parquet row count: 533
- Development sample row count: 397
- Audit table row count: 2,663
- All read-back schemas matched the original schemas
- Training up-day total before and after read-back: 1,075 / 1,075

Task 2: Analytical Questions

Analysis 1: Monthly Increase Ratio

Analysis 1 reproduced the Sprint 3 monthly increase-ratio analysis using the same input records, filtering logic, grouping rules, and missing-value treatment.

The analysis used 1,033,587 minute-level rows.

The results were:

- Rejected rows: 0
- Valid rows: 1,033,587
- Rows with price_change = None: 73,697
- Comparable rows: 959,890
- Monthly results matched: 12 out of 12
- Ratio range: 0.4950 to 0.5023

Month	Increase Count	Comparable Rows	Increase Ratio
1	40,634	81,372	0.4994
2	41,969	84,465	0.4969

3	42,240	84,758	0.4984
4	37,039	73,972	0.5007
5	39,971	80,460	0.4968
6	40,141	80,905	0.4961
7	41,138	81,898	0.5023
8	39,778	80,091	0.4967
9	39,154	78,564	0.4984
10	37,761	75,986	0.4969
11	37,787	75,913	0.4978
12	40,344	81,506	0.4950

Analysis 1 Finding

All monthly increase ratios were close to 0.50. This indicates that the monthly increase result was approximately a coin flip.

The analysis did not provide strong evidence that a specific month consistently produces more stock-price increases. However, it successfully verified that the PySpark implementation reproduced the PA3 baseline correctly.

Analysis 2: Average Daily Change by Calendar Month

Analysis 2 calculated the average daily change_pc for each calendar month using 2,769 daily observations.Analysis 2 Finding

Month	Month Name	Quarter	Days	Average Change	Standard Deviation	Up-Day Percentage	Rank
1	January	Q1	234	0.0724	1.7342	0.5171	7
2	February	Q1	240	0.1092	2.3017	0.4896	6

3	March	Q1	242	-0.2620	2.3688	0.4504	12
4	April	Q2	212	0.1125	2.1067	0.4906	5
5	May	Q2	233	-0.0729	1.8117	0.4592	11
6	June	Q2	232	0.2568	1.8389	0.5259	1
7	July	Q3	240	0.2308	1.5381	0.5542	3
8	August	Q3	229	0.1717	2.3058	0.4629	4
9	September	Q3	226	0.0406	1.8511	0.4779	9
10	October	Q4	223	0.0229	1.5632	0.5336	10
11	November	Q4	222	0.2568	2.2363	0.4775	1
12	December	Q4	235	0.0439	1.6050	0.5106	8

June and November had the highest average daily change at approximately +0.26%.

March had the lowest average daily change at approximately -0.26%.

However, the standard deviations ranged from approximately 1.5 to 2.4 percentage points. These standard deviations were much larger than the differences between the monthly averages.

Therefore, the results did not provide strong evidence of a stable seasonal pattern.

The labeled chart for this analysis is saved as:

```
None
pa4/docs/analysis2_avg_change_by_month.png
```

Task 3: Sampling and Final-Sprint Handoff

The team selected Path B, prediction preparation.

The target variable is target_up, which indicates whether the stock price increased on the next trading day.

The prediction is made at the end of the current trading day. Outcome-revealing fields were excluded from the feature list.

Model Features

The feature list is:

```
None
ret_1d
ret_lag1
ret_lag2
ret_mean_90
ret_std_90
range_pct
vol_ratio_20
month
dow
```

Train/Test Split

The team produced 2,663 labelled rows.

The following rows were removed:

- 1 row for the final day because there was no next day
- 16 rows with a flat next closing price
- 89 warm-up rows missing enough history for the 90-day and 20-day features

The time split produced:

- Training rows: 2,129
- Purged boundary row: 1
- Testing rows: 533

The first test date was February 7, 2024. The check that the latest training date occurred before the earliest testing date passed.

Development Sample

The development sample was created from the training data only.

Sampling method:

```
None
sample(withReplacement=False, fraction=0.2, seed=131)
```

The training population contained 2,129 rows. The development sample contained 397 rows, representing 18.6% of the training data.

Target Group	Source Count	Source Percentage	Sample Count	Sample Percentage
Down / 0	1,054	49.51%	193	48.61%
Up / 1	1,075	50.49%	204	51.39%

The source and sample target proportions were similar. Year proportions changed by no more than 1.80 percentage points.

Handoff Outputs

The prepared outputs were saved in persistent team storage:

```
None
pa4/data/prepared/daily_full.parquet
pa4/data/prepared/train.parquet
pa4/data/prepared/test.parquet
pa4/data/prepared/dev_sample_train.parquet
pa4/out/analysis_1.csv
pa4/out/analysis_2.csv
pa4/out/quality_checks.csv
pa4/out/sample_comparison.csv
pa4/out/benchmark.csv
```

The daily Parquet file contains 2,769 rows.

The training Parquet file contains 2,129 rows.

The testing Parquet file contains 533 rows.

The development sample contains 397 rows.

Both analysis files contain 12 monthly result rows.

Task 4: PySpark and DuckDB Benchmark

Task 4 compared the PA3 monthly analysis using PySpark and DuckDB.

Both tools used:

- The same input records
- The same TSV input format
- The same filtering rules
- The same grouping rules
- The same ordering
- The same numeric verification rules

The benchmark input contained 1,033,587 rows and was 87,079,423 bytes.

Benchmark ResultsBenchmark Summary

Implementation	Environment	Input Rows	Input Bytes	Format	Run	Time	Output Rows	Verified
PySpark	Colab	1,033,587	87,079,423	TSV	1	3.6132 sec	12	True
PySpark	Colab	1,033,587	87,079,423	TSV	2	2.8331 sec	12	True
PySpark	Colab	1,033,587	87,079,423	TSV	3	2.3469 sec	12	True
DuckDB	Colab	1,033,587	87,079,423	TSV	1	0.6216 sec	12	True
DuckDB	Colab	1,033,587	87,079,423	TSV	2	0.5415 sec	12	True
DuckDB	Colab	1,033,587	87,079,423	TSV	3	0.5593 sec	12	True

DuckDB results:

- Mean: 0.574133 seconds
- Median: 0.5593 seconds
- Standard deviation: 0.042060 seconds
- Minimum: 0.5415 seconds
- Maximum: 0.6216 seconds

PySpark results:

- Mean: 2.931067 seconds
- Median: 2.8331 seconds
- Standard deviation: 0.638809 seconds
- Minimum: 2.3469 seconds
- Maximum: 3.6132 seconds

Benchmark Interpretation

DuckDB was faster than PySpark for this specific Colab environment and timing boundary.

The DuckDB median was 0.5593 seconds, while the PySpark median was 2.8331 seconds. Therefore, PySpark was approximately 5.1 times slower than DuckDB by median runtime.

DuckDB also showed lower run-to-run variation.

All six runs generated 12 output rows, and every run was verified successfully.

These measurements describe one machine, one environment, and one workload. They do not prove that DuckDB is always faster than Spark. They also do not measure Spark's potential performance in a distributed multi-machine environment.

Failures Encountered and Diagnosis

One issue occurred when a team member attempted to push Task 4 directly from Google Colab. GitHub returned an authentication error because the required username, email, and personal access token had not been configured.

The team recommended downloading the repository locally and pushing from an IDE or terminal. Another option was to configure the GitHub credentials and personal access token in Google Colab.

Another issue was determining whether Sprint 4 should use the full dataset or only the sample. The team clarified that Sprint 4 used one stock sample to prepare and verify the pipeline for Sprint 5. The same functions can later be applied to the full dataset after filtering by stock name.

Limitations

The detailed analyses were performed using one stock, APOLLOHOSP. Therefore, the findings should not automatically be generalized to every Nifty100 stock.

The monthly increase ratios were close to 50%, and the differences in average monthly change were small compared with their standard deviations. Therefore, the current results do not establish a strong seasonal pattern.

The benchmark results depend on the Colab environment, hardware, caching, software versions, input format, and timing boundaries. The results should be interpreted as measurements for this specific workload rather than universal performance comparisons.

Team Contributions and AI-Use Disclosure

Team Members

- Aydan Kaneshiro – Data Engineer
- Devarsh Shroff – Data Engineer
- Rishil Nilesh Patel – Data Storyteller
- Vakeesan Moorthy – Data Engineer
- Jeremy Lim – Project Manager

Team Contributions

- Vakeesan Moorthy: Completed Task 1 daily data preparation and derived variables.
- Devarsh Shroff: Completed Task 2 analytical questions and Task 3 sampling and final-sprint handoff.
- Aydan Kaneshiro: Completed Task 4 PySpark and DuckDB benchmark.
- Rishil Nilesh Patel: Prepared the Sprint 4 technical report and documentation.
- Jeremy Lim: Managed project coordination and sprint requirements.

AI-Use Disclosure

AI tools were used to brainstorm possible stock-model features, check PySpark syntax, debug code and GitHub commands, and organize the report.

The team reviewed the suggestions and remains responsible for the submitted code, results, explanations, and ability to explain the project.